

Optional Day 3 Breakout: Build API Endpoints

This optional 30-minute breakout is a team-choice lab. Each team chooses **one** challenge, copies only that challenge's starter files into its local quickstart project, and uses the included failing tests as the target. The baseline quickstart source does not include these lab endpoints.

Objectives

1. Build a Spring Boot endpoint from an executable test contract.
2. Practice request mapping, JSON responses, and appropriate HTTP status codes.
3. Use constructor injection when a controller delegates to a service.
4. Compare the team's implementation with a complete reference solution.

Choose One Challenge

Challenge	Level	Main concepts
1. Square	Basic	Query parameter and JSON response
2. Text details	Basic	Path variable and text transformations
3. Shipping estimate	Intermediate	Controller/service separation
4. Tax calculation	Intermediate	Deterministic business rules
5. Inventory API	Advanced stretch	Collection, lookup, create, and errors
6. Order pricing	Advanced stretch	POST body, records, validation, and totals

30-Minute Team Workflow

- **0-3 minutes:** Form a team, choose exactly one challenge, and assign a driver and navigator.
- **3-5 minutes:** From the `quickstart-project` root, copy that challenge's `start/src` tree with the exact command in its section.
- **5-8 minutes:** Run the challenge test. Confirm the starter compiles and the test fails at a TODO behavior.
- **8-22 minutes:** Implement only the TODOs. Rotate the driver halfway through if practical.
- **22-27 minutes:** Run the focused test until it passes, then run `mvn test` to check the baseline tests too.
- **27-30 minutes:** Compare with `solution/`, prepare one observation, and share it with the class.

No dependency or `pom.xml` change is needed. All files use package `com.apidev.quickstart`. Do not copy more than one challenge into the project at a time.

Copy and Run Pattern

Run every command from the local `quickstart-project` root. The challenge sections provide ready-to-run values:

```
cp -R "breakouts/build-api-endpoints/<challenge>/start/src/." "src/"
mvn -Dtest=<ChallengeTest> test
```

After completing the TODOs:

```
mvn -Dtest=<ChallengeTest> test
mvn test
mvn spring-boot:run
```

Run the section's `curl` command in a second terminal while the application is running. Stop it with `Ctrl+C`. To inspect the reference implementation, copy it over the starter files and rerun the same focused test:

```
cp -R "breakouts/build-api-endpoints/<challenge>/solution/src/." "src/"
mvn -Dtest=<ChallengeTest> test
```

If this project is a clean Git checkout, restore it after the lab with `git restore src` and remove the copied lab files shown by `git status --short src`.

Challenge 1: Square

Goal: Implement `SquareController` only.

```
cp -R "breakouts/build-api-endpoints/01-square/start/src/." "src/"
mvn -Dtest=SquareControllerTest test
```

Expected request and response:

```
curl "http://localhost:8080/api/v1/math/square?number=7"
# {"input":7,"square":49}
```

Copy-Paste AI Prompt

```
I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me
complete the optional Square endpoint
without adding dependencies or changing tests. Read
src/main/java/com/apidev/quickstart/controller/SquareController.java and
src/test/java/com/apidev/quickstart/controller/SquareControllerTest.java.
Implement only the explicit TODO.
```

```
GET /api/v1/math/square?number=7 must return HTTP 200 JSON with input 7 and square 49. Keep package com.apidev.quickstart, preserve the existing class and method signatures, use Spring MVC fundamentals, and match the existing four-space Java formatting. Do not use Lombok. Explain the small change, then run mvn -Dtest=SquareControllerTest test and report the result.
```

Challenge 2: Text Details

Goal: Implement `TextDetailsController` only.

```
cp -R "breakouts/build-api-endpoints/02-text-details/start/src/." "src/"
mvn -Dtest=TextDetailsControllerTest test
```

Expected request and response:

```
curl "http://localhost:8080/api/v1/text/spring/details"
# {"original":"spring","length":6,"uppercase":"SPRING"}
```

Copy-Paste AI Prompt

```
I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me complete the optional Text Details endpoint without adding dependencies or changing tests. Read src/main/java/com/apidev/quickstart/controller/TextDetailsController.java and src/test/java/com/apidev/quickstart/controller/TextDetailsControllerTest.java. Implement only the explicit TODO. GET /api/v1/text/{word}/details must return HTTP 200 JSON fields original, length, and uppercase; for spring the values are spring, 6, and SPRING. Keep package com.apidev.quickstart, preserve existing signatures, use a path variable and Spring MVC fundamentals, and match the existing four-space formatting. Do not use Lombok. Explain the small change, then run mvn -Dtest=TextDetailsControllerTest test and report the result.
```

Challenge 3: Shipping Estimate

Rules: Base cost is `weightKg * 2.50`. Standard has no priority fee; express adds `7.50`.

```
cp -R "breakouts/build-api-endpoints/03-shipping-estimate/start/src/" "src/"
mvn -Dtest=ShippingControllerTest test
```

Expected requests and responses:

```
curl "http://localhost:8080/api/v1/shipping/estimate?weightKg=2&priority=standard"
#
{"weightKg":2.0,"priority":"standard","baseCost":5.0,"priorityFee":0.0,"totalCost":5.0}

curl "http://localhost:8080/api/v1/shipping/estimate?weightKg=2&priority=express"
# same fields with priorityFee 7.5 and totalCost 12.5
```

Copy-Paste AI Prompt

I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me complete the optional Shipping Estimate challenge without adding dependencies or changing tests. Read ShippingController.java, ShippingCostService.java, and ShippingControllerTest.java under src. Implement only their explicit TODOs. Build GET /api/v1/shipping/estimate with weightKg and priority query parameters. Base cost is weightKg times 2.50; standard adds 0.00 and express adds 7.50. Return JSON fields weightKg, priority, baseCost, priorityFee, and totalCost. Keep the service calculation separate from the controller response mapping, retain constructor injection, preserve package com.apidev.quickstart and existing signatures, and do not use Lombok. Use deterministic logic and match existing formatting. Explain the changes, then run mvn -Dtest=ShippingControllerTest test and report the result.

Challenge 4: Tax Calculation

Rules: Standard rate is 0.20; reduced rate is 0.10. Tax is $\text{subtotal} * \text{rate}$; total is subtotal plus tax.

```
cp -R "breakouts/build-api-endpoints/04-tax-calculation/start/src/" "src/"
mvn -Dtest=TaxControllerTest test
```

Expected requests and responses:

```
curl "http://localhost:8080/api/v1/tax/calculate?
subtotal=100&region=standard"
#
{"subtotal":100.0,"region":"standard","rate":0.2,"tax":20.0,"total":120.0}

curl "http://localhost:8080/api/v1/tax/calculate?
subtotal=100&region=reduced"
# same fields with rate 0.1, tax 10.0, and total 110.0
```

Copy-Paste AI Prompt

I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me complete the optional Tax Calculation challenge without adding dependencies or changing tests. Read TaxController.java, TaxCalculationService.java, and TaxControllerTest.java under src. Implement only their explicit TODOs. Build GET /api/v1/tax/calculate with subtotal and region query parameters. The standard rate is 0.20 and reduced is 0.10. Calculate tax as subtotal times rate and total as subtotal plus tax. Return JSON fields subtotal, region, rate, tax, and total. Keep business logic in the service, response mapping in the controller, constructor injection, package com.apidev.quickstart, and all existing signatures. Do not use Lombok. Explain the changes, then run mvn -Dtest=TaxControllerTest test and report the result.

Challenge 5: Inventory API

Goal: Complete an in-memory API with seeded items. Missing SKUs return 404; a successful create returns 201 and a Location header; duplicate SKUs return 409. There is no persistence.

```
cp -R "breakouts/build-api-endpoints/05-inventory-api/start/src/." "src/"
mvn -Dtest=InventoryControllerTest test
```

Expected behavior:

```
curl "http://localhost:8080/api/v1/inventory"
curl "http://localhost:8080/api/v1/inventory/B00K-1"
curl -i "http://localhost:8080/api/v1/inventory/UNKNOWN" # HTTP 404
curl -i -X POST "http://localhost:8080/api/v1/inventory" \
```

```
-H "Content-Type: application/json" \
-d '{"sku":"PEN-1","name":"API Pen","quantity":20}' # HTTP 201
# Repeating the POST for PEN-1 returns HTTP 409.
```

The collection starts with **BOOK-1** (**API Fundamentals**, quantity 8) and **MUG-1** (**Developer Mug**, quantity 12).

Copy-Paste AI Prompt

I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me complete the optional Inventory API stretch challenge without adding dependencies, persistence, or external calls, and without changing tests. Read InventoryItem.java, InventoryService.java, InventoryController.java, and InventoryControllerTest.java under src. Implement only explicit TODOs. Preserve the seeded BOOK-1 and MUG-1 records. GET /api/v1/inventory returns the in-memory collection; GET /api/v1/inventory/{sku} returns an item or HTTP 404; POST /api/v1/inventory accepts an InventoryItem, stores a new SKU, and returns HTTP 201 with the item and Location /api/v1/inventory/{sku}; duplicate SKUs return HTTP 409. Keep package com.apidev.quickstart, the existing record and method signatures, synchronized in-memory service methods, and constructor injection. Use only Spring Boot fundamentals and no Lombok. Explain the changes, then run mvn -Dtest=InventoryControllerTest test and report the result.

Challenge 6: Order Pricing

Rules: Sum every line's **unitPrice** * **quantity**. Orders totaling at least 10 units receive a 10% quantity discount. Apply 8% tax after the discount. Round monetary response values to two decimal places. Invalid requests, including an empty item list, return **400** through Jakarta Bean Validation.

```
cp -R "breakouts/build-api-endpoints/06-order-pricing/start/src/." "src/"
mvn -Dtest=OrderControllerTest test
```

Expected behavior:

```
curl -X POST "http://localhost:8080/api/v1/orders" \
-H "Content-Type: application/json" \
-d '{"items":[{"sku":"BOOK-1","unitPrice":20.00,"quantity":2},
{"sku":"PEN-1","unitPrice":5.00,"quantity":8}]}'
#
```

```
{"subtotal":80.00,"totalQuantity":10,"quantityDiscount":8.00,"tax":5.76,"total":77.76}
```

```
curl -i -X POST "http://localhost:8080/api/v1/orders" \
-H "Content-Type: application/json" -d '{"items":[]}' # HTTP 400
```

Copy-Paste AI Prompt

I am working in a Java 21 Spring Boot 3.2.5 quickstart project. Help me complete the optional Order Pricing stretch challenge without adding dependencies, persistence, or external calls, and without changing tests. Read `OrderController.java`, `OrderPricingService.java`, `OrderRequest.java`, `OrderLineItem.java`, `OrderResponse.java`, and `OrderControllerTest.java` under `src`. Implement only explicit TODOs. `POST /api/v1/orders` accepts multiple line items with `sku`, `unitPrice`, and `quantity`. Calculate each line subtotal and the order subtotal, sum `totalQuantity`, apply a 10% quantity discount when `totalQuantity` is at least 10, apply 8% tax to the discounted subtotal, and return subtotal, `totalQuantity`, `quantityDiscount`, `tax`, and `total`. Round money to two decimals with `HALF_UP`. Keep the existing Jakarta validation so malformed values or an empty list return HTTP 400. Preserve package `com.apidev.quickstart`, records, signatures, constructor injection, and formatting. Do not use Lombok. Explain the changes, then run `mvn -Dtest=OrderControllerTest test` and report the result.

Getting Stuck?

Read the focused test first: it is the executable contract. Ask another team member to explain the request-to-service data flow. If the team is still blocked, compare the same relative files under the challenge's `solution/src` tree. The solution has no TODOs and uses the same files and tests as the starter.